Being Linux Torvalds

antirez 26 days ago. 63879 views.

(This blog post was adapted from the transcription obtained from my YouTube video at https://www.youtube.com/watch?v=16lxgYeVZqs)

When Linus Torvalds developed the first Linux kernel, he had studied the Minix sources, he had studied computer architecture, he had the base knowledge needed, and he was obviously a very brilliant programmer. But that operation of writing a minimal yet working Unix kernel for the 386 (at the beginning Linux was, let's say, mono-architecture) was something within the reach of many other programmers and students. Many in the sense of, I don't know, 0.1%, one in a thousand, one in ten thousand. Obviously most people are not able to do this kind of feat, but a lot of people are. If you look at Hacker News in the latest years, you'll see how many projects of kernels written in C, microkernels implemented from scratch, kernels written in Rust, kernels made in all sauces and manners, small Unix systems created vertically for the Raspberry Pi, operating systems for the ESP32 and so forth. Writing a kernel is not something within everybody's reach, but it is something that many can complete, if they put enough effort into it. Then, of course, not everybody will do it well. He is a genius programmer, without any doubt, so he did it better.

And yet, of Linus there is only one. This implementative capacity of his, in fact, would not tell us much about him: what we should focus on, instead, is what happened later.

He stopped writing code

Among the maintainers of the famous open source projects, he was one of the very few that, very early in the history of the development of Linux, almost completely stopped writing code in order to concentrate on the leading of the project. On being the leader, the coordinator, the single mind holding the clarity about what the goals of the project must be, and so on. And this is a rare thing. Many maintainers (myself included, for a long time) continue instead to implement things directly, to not delegate much, and so forth.

This also starts from a different idea of software. Linux, necessarily, had to grow immeasurably: it is in the quality itself of a kernel that wants to embrace many devices, platforms, subsystems, and to continuously adapt to the times, to the needs of the new software, to the hardware that comes out little by little. So this was not a mistake. Redis, on the contrary, could remain something self contained. The other day I received a pull request on linenoise from Dr. Richard Hipp of SQLite: he too aimed at stability, at minimalism, at performances, but always keeping the code base very small, and he continued to write code for a very long time. Linus, instead, no. He understood immediately that he had to donate his time to something that was more important, for a project destined to become very big compared to what is the implementative capacity of a single person.

So he became the project leader, the one that owns the ideas, the direction. And what is it that Linus does, then? He does not look at every patch line by line, every time. Of course it also happens to him to look deeply into a single implementation, in order to understand what is going on. It happened to him, over the years, to write some new subsystem, or even to rewrite one: I think he did it once with the USB layer, many years ago, and he did it with the virtual file system, that at some point I believe he reimplemented, changing the structure of the inodes and of the inode cache, and he did it for several other reasons. From time to time he continued to program, when he created Git, and so forth. But for the most part he does not look at the patches singularly, in detail, line after line: he communicates with the maintainers of the subsections, and understands if a given feature or a given direction is, or is not, a road to take.

So, to say it in Brooks' terms, in Mythical Man Month terms, Linus holds the design concepts of the kernel, and continues to dialogue with everybody below him in the hierarchy of the kernel so that the kernel goes towards a certain direction. So that the developments go towards a certain direction, both from the implementative point of view (how these developments are implemented, what is the quality, what is the implementative idea in the very way the code is written), and from the design point of view: what is it that we want to do, what we don't want, what is the best strategy for the modules, for the scheduler, for the hardware support, for the integration of Rust or not. All this stuff here.

Now, I believe that this was the real genius of Linus. He is not just a very brilliant programmer: there are others. He is also a maintainer, an incredible designer, and one capable of handling a huge project ideas and structure in a coherent way, dialoguing with many other people. This thing is not for everybody.

We are Linus, now

Now, when we program with the artificial intelligences, we are exactly that same thing. We are Linus Torvalds, not always with the talent that he has, but the role we should assume, in the projects where we don't do the review of every line of the code, is exactly of that type. It is exactly the role that he has.

Only, the thing is simpler to dominate: unless we use a lot of agents in parallel, it is substantially simpler to dominate than a multitude of patches arriving from different ways. But it is much faster. It is as if, instead of interacting with a team composed of many people at human speed, we interacted with a team composed of one, two, three people, based on how many parallel branches of our project we are developing in that moment, but that are much faster, so they give us immediately a much faster feedback. This slightly changes the modality of the work, but in my opinion for the better: it is easier, less context switching, fewer people to deal with, many fewer problems due to the character, the attitude, and so forth.

So, if we think that this role is important, we must not think that automatic programming is "I put the prompt, and the thing writes". Vibe coding is a wrong idea of what automatic programming is, and of what automatic programming will be for the majority of people. Vibe coding is a very interesting thing for who does not have technological abilities and wants anyway to have an impact on the construction of their own tools, and so forth: so, welcome, because it democratizes the possibilities. But it is not that.

Automatic programming, instead, in the hands of people that are expert technicians, or expert programmers, expert designers, expert software architects, is to assume the role of Linus, with the agents and the LLMs assuming the role of the different maintainers of the different subsystems. And since not everybody is able to do it so well, automatic programming as well has need of talents that talk with the agents, that check the ideas, that know which are the implementations to do and the ones not to do, the way of communicating with the agents in order to make them do the best work, putting there those design hints that a great programmer intuits, that a good programmer intuits and manages to precompute.

So automatic programming, when it is done well, means to assume the role of Linus. And this thing can be done well, it can be done badly, it can be understood, or it can instead be debased. And it is also something that needs training, that needs to be learned, exactly as Linus had to learn it: he surely had an innate talent for this, but he passed from "I implement everything" to that capability of handling a symphony, of being the orchestra director.

That, for me, is the lesson of Linus, and it is one that should immediately be used as an argument of contrast for those that say that, well, with the LLMs programming is easy for everybody.